



THANDIWE THABETHE

ST10494913

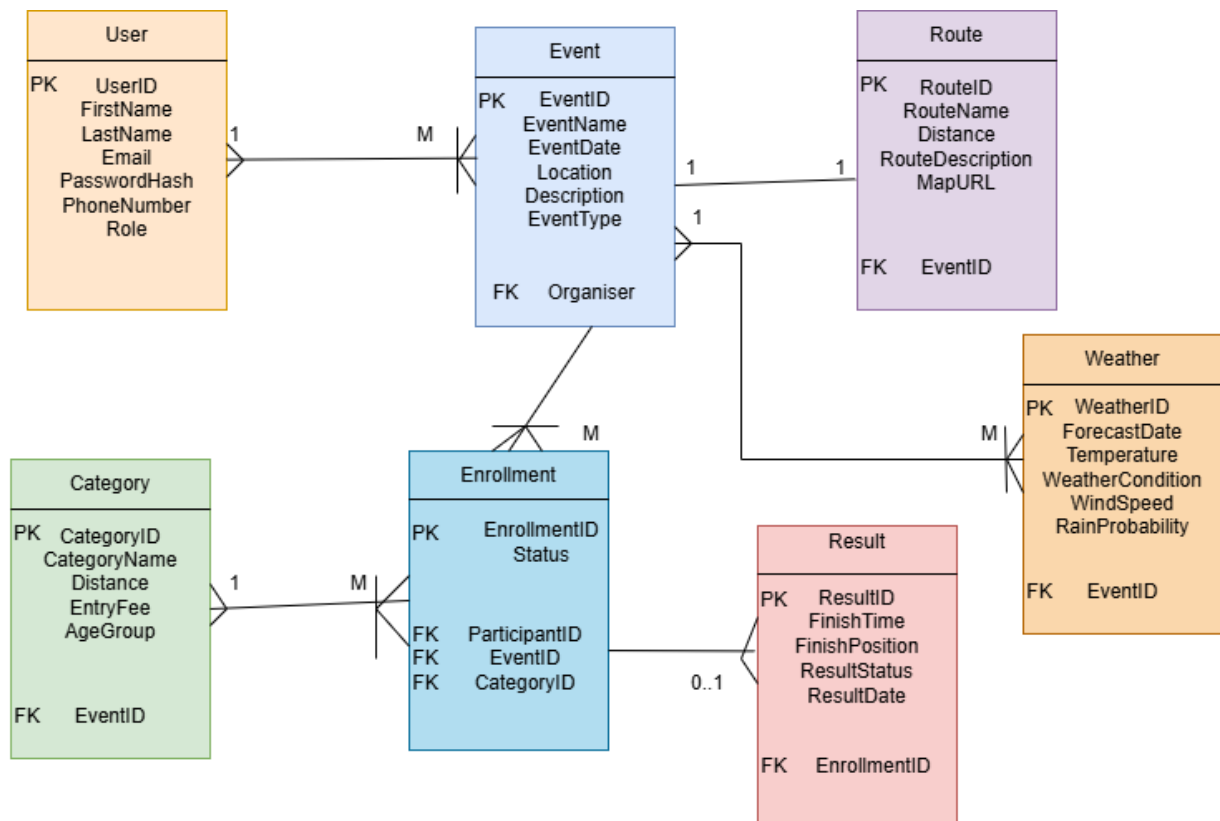


PROG

Table of Contents

Raceday – ERD	2
Create database Raceday.....	5
Table User	5
Table Event.....	6
Table Category	6
Table Enrollment	7
Table Result	8
Table Route	9
Table Weather	10
DISPLAY ORGANISERS AND THEIR EVENTS	10
DISPLAY PARTICIPANT RESULTS	11
DISPLAY EVENT ROUTES	11
DISPLAY EVENT WEATHER	11

Raceday – ERD



HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either an Organiser or Participant.	None (Public)	{ firstName, lastName, email, password, phoneNumber, role }	201 Created – User account created. 400 Bad Request – Invalid or missing information. 409 Conflict – Email already exists.
POST	/api/auth/login	Authenticates a registered user and allows access to protected system features.	None (Public)	{ email, password }	200 OK – Login successful and token returned. 401 Unauthorized – Invalid email or password.
Get	/api/users/me	Returns the profile information of the currently logged-in user.	Any logged-in user	None	200 OK – User profile returned. 401 Unauthorized – User is not logged in.
PUT	/api/users/me	Allows the currently logged-in user to update their own profile information.	Any logged-in user	{ firstName, lastName, phoneNumber }	200 OK – Profile updated successfully. 400 Bad Request – Invalid data. 401 Unauthorized – User is not logged in.
GET	/api/events	Retrieves a list of all available RaceDay events.	None (Public)	None	200 OK – List of events returned.
GET	/api/events/{id}	Retrieves detailed information about one specific event.	None (Public)	None	200 OK – Event returned. 404 Not Found – Event does not exist.
POST	/api/events	Creates a new event. The event is linked to the logged-in Organiser.	Organiser	{ eventName, eventDate, location, description, eventType }	201 Created – Event created. 400 Bad Request – Invalid information. 401 Unauthorized – Not logged in. 403 Forbidden – User is not an Organiser.
PUT	/api/events/{id}	Updates an existing event.	Organiser	{ eventName, eventDate, location, description, eventType }	200 OK – Event updated. 403 Forbidden – User is not authorised. 404 Not Found – Event does not exist.
DELETE	/api/events/{id}	Deletes an existing event.	Organiser	None	204 No Content – Event deleted. 403 Forbidden – Not authorised. 404 Not Found – Event does not exist.
GET	/api/events/{eventId}/categories	Retrieves all categories belonging to a specific event.	None (Public)	None	200 OK – Categories returned. 404 Not Found – Event does not exist.

GET	/api/events/{eventId}/categories	Retrieves all categories belonging to a specific event.	None (Public)	None	200 OK – Categories returned. 404 Not Found – Event does not exist.
GET	/api/categories/{id}	Retrieves one specific category.	Text None (Public)	None	200 OK – Category returned. 404 Not Found – Category does not exist.
POST	/api/events/{eventId}/categories	Creates a new category for an event.	Organiser	{ categoryName, distance, entryFee, ageGroup }	201 Created – Category created. 400 Bad Request – Invalid data. 403 Forbidden – Not authorised.
PUT	/api/categories/{id}	Updates an existing category.	Organiser	{ categoryName, distance, entryFee, ageGroup }	200 OK – Category updated. 404 Not Found – Category does not exist.
DELETE	/api/categories/{id}	Deletes a category from an event.	Organiser	None	204 No Content – Category deleted. 403 Forbidden – Not authorised. 404 Not Found – Category does not exist.
POST	/api/events/{eventId}/enrollments	Creates an enrolment for the logged-in Participant and selected event category.	Participant	{ categoryId }	201 Created – Enrolment created. 400 Bad Request – Invalid data. 403 Forbidden – User is not a Participant. 409 Conflict – Participant is already enrolled.
GET	/api/users/me/enrollments	Retrieves all enrolments belonging to the currently logged-in Participant.	Participant	None	200 OK – Participant enrolments returned.
DELETE	/api/enrollments/{id}	Cancels or deletes an enrolment belonging to the logged-in Participant.	Participant	None	204 No Content – Enrolment deleted. 403 Forbidden – Not authorised. 404 Not Found – Enrolment does not exist.
GET	/api/events/{eventId}/route	Retrieves the route information for a specific event.	None (Public)	None	200 OK – Route returned. 404 Not Found – Route or event does not exist.
POST	/api/events/{eventId}/route	Creates route information for an event.	Organiser	{ routeName, distance, routeDescription, mapUrl }	201 Created – Route created. 400 Bad Request – Invalid data. 403 Forbidden – Not authorised
PUT	/api/routes/{id}	Updates the route information for an event.	Organiser	{ routeName, distance, routeDescription, mapUrl }	200 OK – Route updated. 404 Not Found – Route does not exist.
DELETE	/api/routes/{id}	Deletes route information from an event.	Organiser	None	204 No Content – Route deleted. 403 Forbidden – Not authorised. 404 Not Found – Route does not exist.
GET	/api/events/{eventId}/weather	Retrieves weather or forecast information associated with a specific event.	None (Public)	None	200 OK – Weather information returned. 404 Not Found – Event does not exist.

Text is not SVG - cannot display

Create database Raceday

```
CREATE DATABASE RaceDay;
```

Table User

```
CREATE TABLE User
```

```
( UserID INT IDENTITY(1,1) NOT NULL,  
  FirstName NVARCHAR(50) NOT NULL,  
  LastName NVARCHAR(50) NOT NULL,  
  Email NVARCHAR(100) NOT NULL,  
  PasswordHash NVARCHAR(255) NOT NULL,  
  PhoneNumber NVARCHAR(20) NOT NULL,  
  Role NVARCHAR(20) NOT NULL  
  CONSTRAINT CK_User_Role CHECK (Role IN ('Organiser', 'Participant')),  
  CONSTRAINT PK_User PRIMARY KEY (UserID),  
  CONSTRAINT UQ_User_Email UNIQUE (Email) );
```

```
INSERT INTO [User] (
```

```
  FirstName,  
  LastName,  
  Email,  
  PasswordHash,  
  PhoneNumber,  
  Role )
```

```
VALUES
```

```
( 'Lerato', 'Mokoena', 'lerato.organiser@raceday.co.za', CONVERT(VARCHAR(64),  
  HASHBYTES('SHA2_256', 'Lerato123!'), 2), '0825551001', 'Organiser' ), ( 'Sipho', 'Naidoo',  
  'sipho.organiser@raceday.co.za', CONVERT(VARCHAR(64), HASHBYTES('SHA2_256',  
  'Sipho123!'), 2), '0835551002', 'Organiser' ), ( 'Thandiwe', 'Molefe',  
  'thandiwe.participant@example.com', CONVERT(VARCHAR(64), HASHBYTES('SHA2_256',  
  'Thandi123!'), 2), '0845551003', 'Participant' ), ( 'Kabelo', 'Dlamini',  
  'kabelo.participant@example.com', CONVERT(VARCHAR(64), HASHBYTES('SHA2_256',  
  'Kabelo123!'), 2), '0725551004', 'Participant' ), ( 'Ayanda', 'Ndlovu',  
  'ayanda.participant@example.com', CONVERT(VARCHAR(64), HASHBYTES('SHA2_256',  
  'Ayanda123!'), 2), '0715551005', 'Participant' ), ( 'Nomsa', 'Khumalo',  
  'nomsa.participant@example.com', CONVERT(VARCHAR(64), HASHBYTES('SHA2_256',  
  'Nomsa123!'), 2), '0765551006', 'Participant' );
```

Table Event

```
CREATE TABLE Event(  
    EventID INT IDENTITY(1,1) NOT NULL,  
    EventName NVARCHAR(150) NOT NULL,  
    EventDate DATETIME2 NOT NULL,  
    Location NVARCHAR(150) NOT NULL,  
    Description NVARCHAR(MAX) NULL,  
    EventType NVARCHAR(50) NOT NULL,  
    OrganiserID INT NOT NULL,  
  
    CONSTRAINT PK_Event  
        PRIMARY KEY (EventID),  
  
    CONSTRAINT FK_Event_Organiser  
        FOREIGN KEY (OrganiserID)  
        REFERENCES [User](UserID),  
  
    CONSTRAINT CK_Event_Type  
        CHECK (EventType IN ('Run', 'Walk', 'Cycle', 'Other')) );
```

```
INSERT INTO Event (  
    EventName,  
    EventDate,  
    Location,  
    Description,  
    EventType,  
    OrganiserID )  
VALUES ( 'Cape Town Community Road Run', '2026-10-18 07:00:00', 'Cape Town, Western  
Cape', 'A community road running event with multiple distance categories for recreational  
and competitive runners.', 'Run', 1 ), ( 'Johannesburg Charity Run', '2026-11-08 07:00:00',  
'Johannesburg, Gauteng', 'A charity road running event supporting local community  
development initiatives.', 'Run', 2 ), ( 'Durban Coastal Cycle Challenge', '2026-11-22  
06:30:00', 'Durban, KwaZulu-Natal', 'A coastal cycling event offering short and long distance  
categories for cyclists.', 'Cycle', 1 );
```

Table Category

```
CREATE TABLE Category (  
    CategoryID INT IDENTITY(1,1) NOT NULL,  
    CategoryName NVARCHAR(100) NOT NULL,  
    Distance DECIMAL(6,2) NOT NULL,  
    EntryFee DECIMAL(10,2) NOT NULL,
```

AgeGroup NVARCHAR(50) NOT NULL,
EventID INT NOT NULL,

CONSTRAINT PK_Category PRIMARY KEY (CategoryID),
CONSTRAINT FK_Category_Event FOREIGN KEY (EventID)
REFERENCES Event(EventID),

CONSTRAINT CK_Category_Distance CHECK (Distance > 0),
CONSTRAINT CK_Category_EntryFee CHECK (EntryFee >= 0),
CONSTRAINT UQ_Category_Event_Name UNIQUE (EventID, CategoryName),
CONSTRAINT UQ_Category_Event_Category UNIQUE (EventID, CategoryID));

INSERT INTO Category (
CategoryName,
Distance,
EntryFee,
AgeGroup,
EventID)

VALUES ('5 KM Fun Run', 5.00, 80.00, 'Open', 1), ('10 KM Road Run', 10.00, 150.00, 'Open',
1), ('21.1 KM Half Marathon', 21.10, 250.00, '18+', 1);

Results

Messages

	CategoryID	CategoryName	Distance	EntryFee	AgeGroup	EventID
1	1	5 KM Fun Run	5.00	80.00	Open	1
2	2	10 KM Road Run	10.00	150.00	Open	1
3	3	21.1 KM Half Marathon	21.10	250.00	18+	1
4	4	5 KM Charity Walk	5.00	60.00	Open	2
5	5	10 KM Charity Run	10.00	140.00	Open	2
6	6	21.1 KM Half Marathon	21.10	220.00	18+	2
7	7	20 KM Coastal Cycle	20.00	180.00	Open	3
8	8	40 KM Coastal Cycle	40.00	300.00	18+	3

Table Enrollment

CREATE TABLE Enrollment (
EnrollmentID INT IDENTITY(1,1) NOT NULL,
ParticipantID INT NOT NULL,
EventID INT NOT NULL,
CategoryID INT NOT NULL,
EnrollmentDate DATETIME2 NOT NULL

CONSTRAINT DF_Enrollment_Date
DEFAULT GETDATE(),

Status NVARCHAR(20) NOT NULL
 CONSTRAINT DF_Enrollment_Status DEFAULT 'Entered',
 CONSTRAINT PK_Enrollment PRIMARY KEY (EnrollmentID),
 CONSTRAINT FK_Enrollment_Participant
 FOREIGN KEY (ParticipantID)

REFERENCES [User](UserID), CONSTRAINT FK_Enrollment_Event FOREIGN KEY (EventID)
 REFERENCES Event(EventID), CONSTRAINT FK_Enrollment_Category FOREIGN KEY
 (EventID, CategoryID) REFERENCES Category(EventID, CategoryID), CONSTRAINT
 CK_Enrollment_Status CHECK (Status IN ('Entered', 'Cancelled', 'Completed')),
 CONSTRAINT UQ_Enrollment_Participant_Event UNIQUE (ParticipantID, EventID));

INSERT INTO Enrollment (ParticipantID, EventID, CategoryID, EnrollmentDate, Status)
 VALUES (3, 1, 2, '2026-08-20 10:30:00', 'Entered'), (4, 1, 3, '2026-08-21 11:15:00', 'Entered'
), (5, 2, 5, '2026-08-22 09:45:00', 'Entered'), (6, 2, 6, '2026-08-23 14:20:00', 'Entered'), (3,
 3, 7, '2026-08-25 08:30:00', 'Entered'), (4, 3, 8, '2026-08-26 13:10:00', 'Entered');

Results

Messages

	EnrollmentID	ParticipantID	EventID	CategoryID	EnrollmentDate	Status
1	1	3	1	2	2026-08-20 10:30:00.0000000	Entered
2	2	4	1	3	2026-08-21 11:15:00.0000000	Entered
3	3	5	2	5	2026-08-22 09:45:00.0000000	Entered
4	4	6	2	6	2026-08-23 14:20:00.0000000	Entered
5	5	3	3	7	2026-08-25 08:30:00.0000000	Entered
6	6	4	3	8	2026-08-26 13:10:00.0000000	Entered

Table Result

CREATE TABLE Result (
 ResultID INT IDENTITY(1,1) NOT NULL,
 EnrollmentID INT NOT NULL,
 FinishTime TIME(0) NULL,
 FinishPosition INT NULL,
 ResultStatus NVARCHAR(20) NOT NULL,
 ResultDate DATETIME2 NOT NULL

CONSTRAINT DF_Result_Date
 DEFAULT GETDATE(),

CONSTRAINT PK_Result PRIMARY KEY (ResultID),
 CONSTRAINT FK_Result_Enrollment
 FOREIGN KEY (EnrollmentID)
 REFERENCES Enrollment(EnrollmentID),

CONSTRAINT UQ_Result_Enrollment UNIQUE (EnrollmentID),

```
CONSTRAINT CK_Result_Position CHECK (FinishPosition IS NULL OR FinishPosition > 0),
CONSTRAINT CK_Result_Status CHECK (ResultStatus IN ('Finished', 'DNF', 'DNS')) );
```

```
INSERT INTO Result ( EnrollmentID, FinishTime, FinishPosition, ResultStatus, ResultDate )
VALUES ( 1, '01:02:35', 18, 'Finished', '2026-10-18 12:30:00' ), ( 2, '01:49:22', 42, 'Finished',
'2026-10-18 13:00:00' );
```

Results		Messages				
	ResultID	EnrollmentID	FinishTime	FinishPosition	ResultStatus	ResultDate
1	1	1	01:02:35	18	Finished	2026-10-18 12:30:00.00000000
2	2	2	01:49:22	42	Finished	2026-10-18 13:00:00.00000000

Table Route

```
CREATE TABLE Route (
RouteID INT IDENTITY(1,1) NOT NULL,
EventID INT NOT NULL,
RouteName NVARCHAR(150) NOT NULL,
Distance DECIMAL(6,2) NOT NULL,
RouteDescription NVARCHAR(MAX) NOT NULL,
MapURL NVARCHAR(255) NULL,
```

```
CONSTRAINT PK_Route PRIMARY KEY (RouteID),
CONSTRAINT FK_Route_Event FOREIGN KEY (EventID)
REFERENCES Event(EventID),
```

```
CONSTRAINT UQ_Route_Event UNIQUE (EventID),
CONSTRAINT CK_Route_Distance CHECK (Distance > 0) );
```

```
INSERT INTO Route ( EventID, RouteName, Distance, RouteDescription, MapURL )
VALUES ( 1, 'Cape Town Community 21 KM Route', 21.10, 'Road route starting in central
Cape Town and continuing through major city roads before returning to the finish area.',
'https://maps.example.com/capetown-community-run' ), ( 2, 'Johannesburg Charity 21 KM
Route', 21.10, 'Road route through selected Johannesburg suburbs with water stations
positioned along the course.', 'https://maps.example.com/johannesburg-charity-run' ), ( 3,
'Durban Coastal 40 KM Cycle Route', 40.00, 'Coastal cycling route starting in Durban and
following selected roads along the KwaZulu-Natal coastline.',
'https://maps.example.com/durban-coastal-cycle' );
```

Results		Messages				
	RouteID	EventID	RouteName	Distance	RouteDescription	MapURL
1	1	1	Cape Town Community 21 KM Route	21.10	Road route starting in central Cape Town and contin...	https://maps.example.com/capetown-community-run
2	2	2	Johannesburg Charity 21 KM Route	21.10	Road route through selected Johannesburg suburbs ...	https://maps.example.com/johannesburg-charity-run
3	3	3	Durban Coastal 40 KM Cycle Route	40.00	Coastal cycling route starting in Durban and following	https://maps.example.com/durban-coastal-cycle

Table Weather

```
CREATE TABLE Weather (  
WeatherID INT IDENTITY(1,1) NOT NULL,  
EventID INT NOT NULL,  
ForecastDate DATE NOT NULL,  
Temperature DECIMAL(5,2) NOT NULL,  
WeatherCondition NVARCHAR(100) NOT NULL,  
WindSpeed DECIMAL(5,2) NOT NULL,  
RainProbability DECIMAL(5,2) NOT NULL,  
  
CONSTRAINT PK_Weather PRIMARY KEY (WeatherID),  
CONSTRAINT FK_Weather_Event FOREIGN KEY (EventID)  
REFERENCES Event(EventID),  
  
CONSTRAINT CK_Weather_WindSpeed CHECK (WindSpeed >= 0),  
CONSTRAINT CK_Weather_RainProbability CHECK (RainProbability >= 0 AND  
RainProbability <= 100),  
CONSTRAINT UQ_Weather_Event_Date UNIQUE (EventID, ForecastDate) );  
  
INSERT INTO Weather ( EventID, ForecastDate, Temperature, WeatherCondition,  
WindSpeed, RainProbability ) VALUES ( 1, '2026-10-16', 18.00, 'Partly Cloudy', 12.00, 20.00 ),  
( 1, '2026-10-17', 17.00, 'Clear', 10.00, 10.00 ), ( 1, '2026-10-18', 19.00, 'Sunny', 8.00, 5.00 ), ( 2, '2026-11-07', 21.00, 'Partly Cloudy', 15.00, 25.00 ), ( 2, '2026-11-08', 23.00, 'Sunny', 11.00, 10.00 ), ( 3, '2026-11-21', 24.00, 'Clear', 18.00, 15.00 ), ( 3, '2026-11-22', 25.00, 'Sunny', 16.00, 5.00 );
```

	WeatherID	EventID	ForecastDate	Temperature	WeatherCondition	WindSpeed	RainProbability
1	1	1	2026-10-16	18.00	Partly Cloudy	12.00	20.00
2	2	1	2026-10-17	17.00	Clear	10.00	10.00
3	3	1	2026-10-18	19.00	Sunny	8.00	5.00
4	4	2	2026-11-07	21.00	Partly Cloudy	15.00	25.00
5	5	2	2026-11-08	23.00	Sunny	11.00	10.00
6	6	3	2026-11-21	24.00	Clear	18.00	15.00
7	7	3	2026-11-22	25.00	Sunny	16.00	5.00

DISPLAY ORGANISERS AND THEIR EVENTS

```
SELECT u.FirstName,  
       u.LastName,  
       e.EventName,  
       e.EventDate,  
       e.Location  
FROM Event e  
JOIN [User] u  
  ON e.OrganiserID = u.UserID
```

```
WHERE u.Role = 'Organiser'  
ORDER BY e.EventDate;
```

DISPLAY PARTICIPANT RESULTS

```
SELECT  
    u.FirstName,  
    u.LastName,  
    e.EventName,  
    c.CategoryName,  
    r.FinishTime,  
    r.FinishPosition,  
    r.ResultStatus  
FROM Result r  
JOIN Enrollment en  
    ON r.EnrollmentID = en.EnrollmentID  
JOIN [User] u  
    ON en.ParticipantID = u.UserID  
JOIN Event e  
    ON en.EventID = e.EventID  
JOIN Category c  
    ON en.CategoryID = c.CategoryID  
ORDER BY r.FinishPosition;
```

DISPLAY EVENT ROUTES

```
SELECT  
    e.EventName,  
    r.RouteName,  
    r.Distance,  
    r.RouteDescription,  
    r.MapURL  
FROM Route r  
JOIN Event e  
    ON r.EventID = e.EventID  
ORDER BY e.EventName;
```

DISPLAY EVENT WEATHER

```
SELECT  
    e.EventName,  
    w.ForecastDate,  
    w.Temperature,  
    w.WeatherCondition,
```

```
w.WindSpeed,  
w.RainProbability  
FROM Weather w  
JOIN Event e  
ON w.EventID = e.EventID  
ORDER BY e.EventName, w.ForecastDate;
```